- `tests/expected-decisions.json`

In Cursor, add these files to the AI context before issuing any prompts. You can do this by opening the files in the editor, by referencing them directly in the prompt using Cursor's file-context feature, or by selecting them from the context panel so the AI can read their contents. Including these files ensures that any proposed policy changes are grounded in the actual ACME model and prevents the AI from inventing entities, actions, or attributes.

Next, open `ai/cursor/starter-prompt.md`. Use the base prompt in that file to initialize the AI session. This prompt establishes the constraints under which AI assistance is allowed, including the requirement to propose test cases and explain policy intent in plain language.

At this point, you can verify that the AI has sufficient context by asking a simple, non-destructive question, such as:

> *"Explain why the sample request results in its expected decision."*

The response should reference the relevant policy files and conditions rather than restating the request in generic terms. If the response introduces entities, actions, or attributes that do not exist in `acme.cedarschema`, stop and recheck which files are included as context.

Providing concrete examples is especially important. Small, representative requests help the AI reason about scope and intent far more effectively than long textual descriptions. For this reason, every policy change discussed in this appendix is grounded in an explicit request and an expected decision.

B.5 A human-in-the-loop authoring workflow

With the repository open and the sample request in place, you can now walk through a complete, *human-in-the-loop policy authoring* workflow. The goal in this section is not to change policy behavior yet, but to see how AI assistance fits into the normal review and iteration process.